Unique Land Parcel Identification for every floor/unit — India's 3D Cadastral Registry Prototype

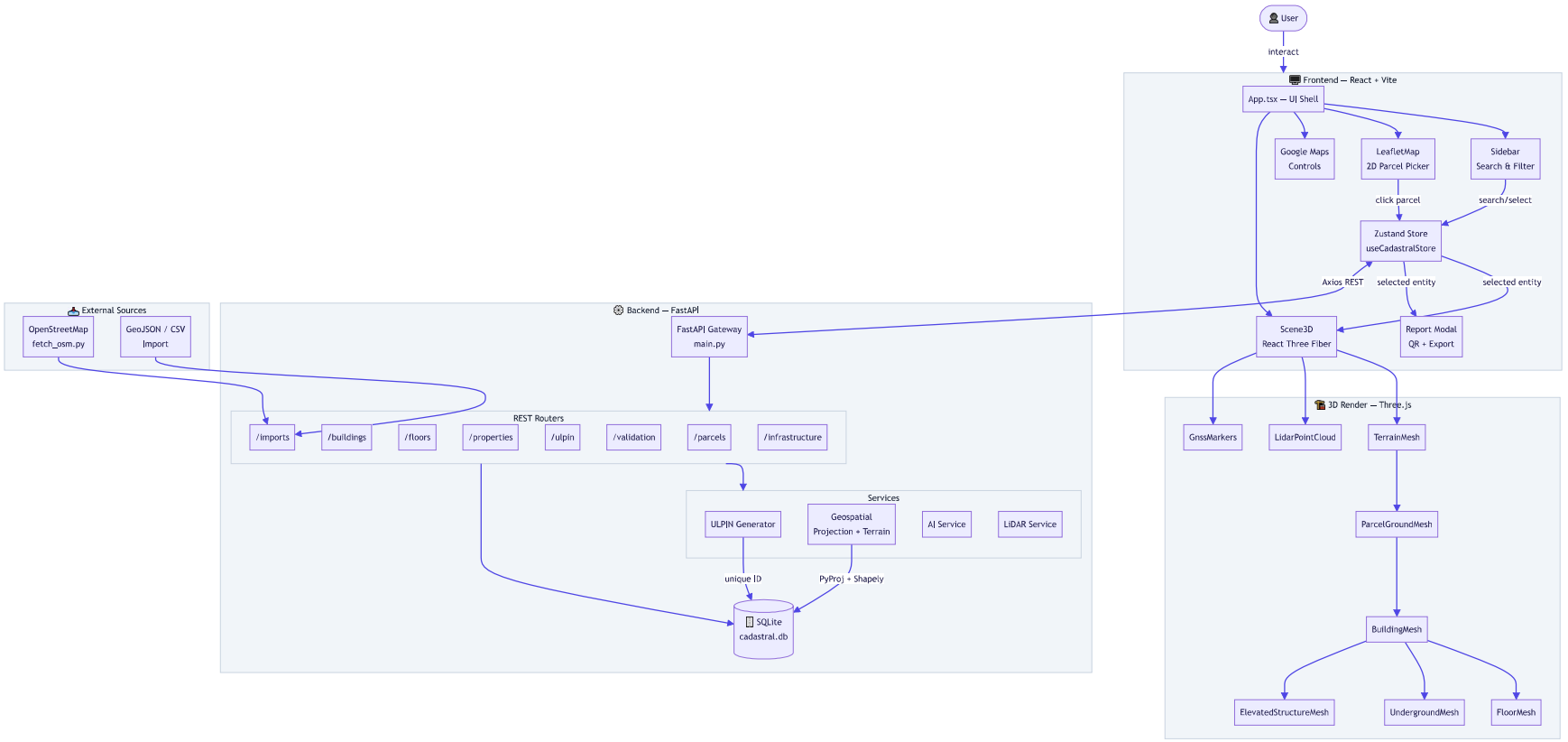
📍 Prayagraj, Uttar Pradesh, India | 🗄️ SQLite Cadastral DB | 🌐 Vercel Deployed | 🏗️ 3D Floor-Level Registry

🧱 Tech Stack

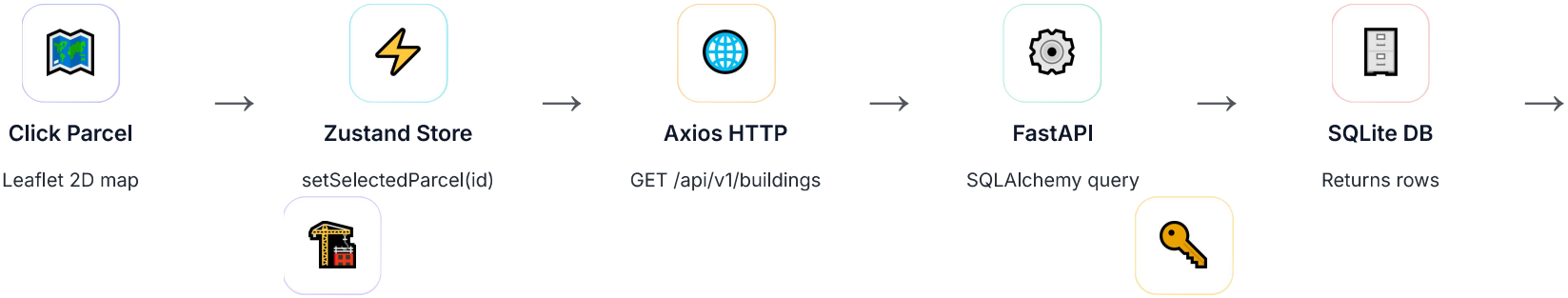
LAYER	TECHNOLOGY	PURPOSE
Frontend Framework	`React 18` + `TypeScript` + `Vite`	SPA shell, bundler
3D Rendering	`Three.js` + `@react-three/fiber` + `Drei`	Interactive 3D building views
2D Map	`Leaflet` + `react-leaflet`	Parcel picker map
Google Maps	Google Maps JS API	Satellite / hybrid base layer
State Management	`Zustand`	Global cadastral store
Styling	`TailwindCSS` + PostCSS	Utility-first UI
HTTP Client	`Axios`	REST API calls to backend
QR Code	`qrcode`	ULPIN QR generation
Backend Framework	`FastAPI` + `Uvicorn`	Async REST API server
ORM	`SQLAlchemy 2.0`	Database models & queries
Database	`SQLite` (cadastral.db)	Parcel / building / floor records
Geometry	`Shapely` + `PyProj`	GIS polygon ops + coord projection

LAYER	TECHNOLOGY	PURPOSE
AI Services	ai_services.py	Internal AI stub for property analysis
LiDAR	lidar_service.py	Simulated point cloud terrain data
Data Validation	`Pydantic v2`	Schema validation for all API payloads
Deployment	`Vercel`	Frontend hosting + vercel.json routing

🔄 System Architecture Flow



Data Flow — Step by Step



Three.js Scene

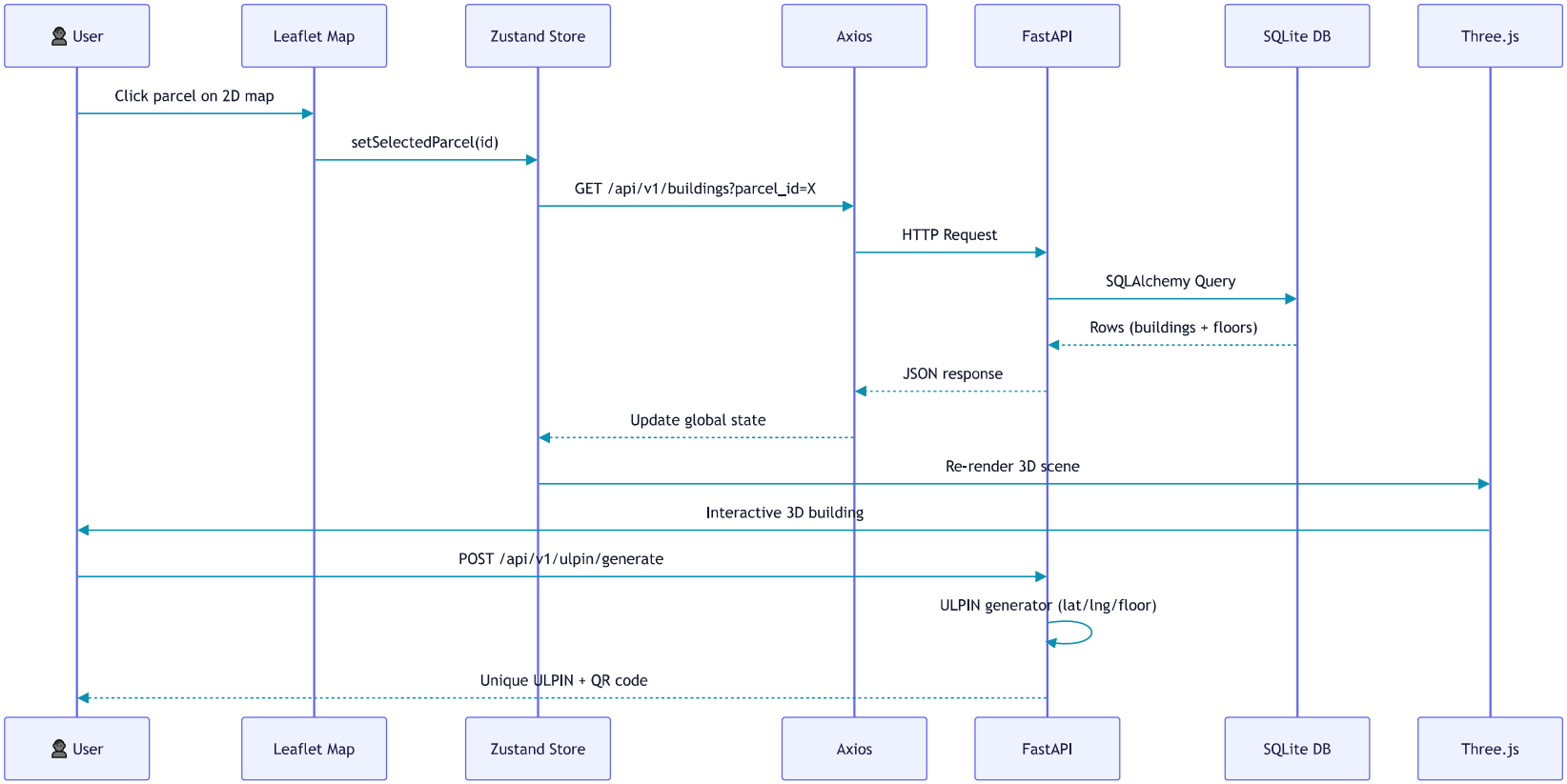
3D render update

→

ULPIN + QR

Report export

📡 Sequence Diagram



🔑 Key Concepts

ULPIN

Unique Land Parcel ID assigned per vertical unit (floor-level). India's cadastral identity for 3D properties.

3D Ladder ("3d lddr")

Vertical stacking model — underground → ground parcel → floors → rooftop → elevated structures.

Cadastral DB

SQLite with SQLAlchemy models for parcels, buildings, floors, properties, and infrastructure records.

Geospatial Projection

WGS84 (GPS) ↔ local metric coordinates via `PyProj`. Enables accurate 3D positioning in Three.js.

LiDAR Service

Simulated point-cloud terrain data for realistic elevation rendering in the 3D scene.

OSM Integration

Imports real-world building footprints from OpenStreetMap via `fetch_osm.py`.

Import Pipeline

Bulk parcel ingestion via GeoJSON, CSV, or OSM data through `/api/v1/imports`.

Validation Layer

Pydantic v2 schemas + dedicated validation API cross-check data integrity before ULPIN assignment.

📂 Module Map

```
3d lddr/
├── frontend/src/
│   ├── three/              → 3D mesh components
│   │   ├── Scene3D.tsx        # canvas root
│   │   ├── BuildingMesh.tsx    # building geometry
│   │   ├── FloorMesh.tsx       # per-floor slabs
│   │   ├── TerrainMesh.tsx     # ground surface
```

```
    │   │   ├── ParcelGroundMesh.tsx# parcel boundary
    │   │   ├── UndergroundMesh.tsx # basement layers
    │   │   ├── ElevatedStructureMesh.tsx
    │   │   ├── LidarPointCloud.tsx # point cloud viz
    │   │   └── GnssMarkers.tsx     # GPS markers
    │   ├── map/
    │   │   └── LeafletMap.tsx      # 2D parcel picker
    │   ├── components/
    │   │   ├── google/             # Google Maps controls
    │   │   ├── layout/             # Sidebar, Header, Details panel
    │   │   ├── report/             # PDF / QR report modal
    │   │   ├── validation/         # Data validation UI
    │   │   └── import/             # File import UI
    │   ├── state/
    │   │   └── useCadastralStore.ts# Zustand global store
    │   └── api/                    # Axios API client

    └── backend/app/
        ├── api/                → FastAPI routers (8 endpoints)
        │   ├── parcels.py
        │   ├── buildings.py
        │   ├── floors.py
        │   ├── properties.py
        │   ├── ulpin.py          # ULPIN generation
        │   ├── validation.py
        │   ├── imports.py        # GeoJSON/OSM/CSV
        │   └── infrastructure.py
        ├── models/             # SQLAlchemy ORM models
        ├── schemas/            # Pydantic schemas
        ├── services/
        │   ├── ai_services.py    # AI analysis stub
```

```
│   └── lidar_service.py  # Simulated LiDAR
├── ulpin/
│   └── generator.py      # ULPIN ID generator
├── geospatial/
│   ├── projection.py     # WGS84 ↔ metric
│   └── terrain.py        # elevation data
├── validation/        # data integrity checks
└── database/          # SQLite session mgmt
```

🔌 API Endpoints

METHOD	ENDPOINT	PURPOSE
GET	/api/v1/parcels	List / query land parcels
GET	/api/v1/buildings	Buildings for a parcel
GET	/api/v1/floors	Floor data for a building
GET	/api/v1/properties	Property unit details
POST	/api/v1/ulpin/generate	Generate ULPIN + QR code
POST	/api/v1/validation	Validate property data
POST	/api/v1/imports	Bulk import GeoJSON / CSV / OSM
GET	/api/v1/infrastructure	Utilities / road infrastructure